

The Layman's Guide to Energy AI Hackathon ML Workflow

A Plain-English Explanation of Every Step

For Team Energy Gladiators

What Are We Building?

We're building a system that **predicts how much energy will be used** when drilling oil wells using a process called hydraulic fracturing (‘‘fracking’’).

The energy comes in three forms: - **Grid electricity** (measured in kWh) - **Diesel fuel** (measured in gallons) - **Compressed Natural Gas / CNG** (measured in MMBTU)

Think of it like this: You want to predict how much gas your car will use on a road trip. You'd look at the distance, the type of road, the car's efficiency, weather, etc. We're doing the same thing, but for industrial drilling operations.

Why This Is More Valuable Than Commercial Software

Companies pay \$3,000-5,000 per year for tools like Spotfire or Tableau. Those tools show you **what already happened** (charts and dashboards).

Our solution is different: - It **predicts what WILL happen** (not just what happened) - It gives you **100 possible outcomes** so you know the range of possibilities - It has an **AI assistant** that can answer questions and adapt to new problems - It's **free** (built with Python and open-source tools)

Bottom line: Spotfire tells you ‘‘here’s what your energy consumption was.’’ Our tool tells you ‘‘here’s what it will be, and here’s how confident we are.’’

The 9 Steps Explained

Step 1: Data Upload & Inspection

What happens: You load two spreadsheets (CSV files) into the system.

- **Training Data:** Information about 1,082 wells where we KNOW how much energy was used
- **Test Data:** Information about 50 wells where we need to PREDICT energy usage

Analogy: The training data is like studying for a test with answer keys. The test data is the actual exam where you apply what you learned.

What to look for: - How many rows (wells) and columns (features) do you have? - Are there any missing values (blank cells)? - What are the column names and data types?

Step 2: Data Cleaning & Imputation

What happens: We fix problems in the data, especially missing values.

‘‘Imputation’’ means: Filling in blank cells with reasonable guesses. - For numbers: We use the **median** (middle value) - safer than average because it's not affected by extreme values - For categories: We use the **mode** (most common value)

Analogy: If a survey respondent left ‘‘age’’ blank, you might fill it with the average age of all respondents. That's imputation.

Tip: Median is often better than average. If most people earn \$50K but one person earns \$10 million, the average salary looks misleadingly high. The median stays realistic.

Step 3: Exploratory Data Analysis (EDA)

What happens: We create charts and statistics to understand our data before building models.

Key Visualizations: 1. **Histograms:** Bar charts showing how values are distributed 2. **Box Plots:** Show the range of values and outliers (unusual data points) 3. **Scatter Plots:** Show relationships between two variables 4. **Correlation Heatmap:** A grid showing how strongly each variable relates to others

Analogy: Before cooking a new recipe, you'd examine all your ingredients - their quantities, freshness, how they might combine. EDA is examining your data ingredients.

Key Finding: Number of Stages and Number of Clusters are the best predictors of energy usage.

Step 4: Feature Engineering

What happens: We create NEW columns from existing data that might help predictions.

Our Engineered Features:

Feature	Formula	What It Captures
Time_Overrun	Actual Time - Estimated Time	Efficiency (positive = took longer than planned)
Total_Pumping_Time	Stages Ã— Stage Time	Total work duration
Clusters_per_Stage	Clusters / Stages	Work intensity at each stage

Analogy: Raw data says "drove 60 mph for 2 hours." Engineered feature: "traveled 120 miles." The new feature is often more useful!

Step 5: Model Training

What happens: We teach the computer to predict energy usage by showing it the training data.

We use Random Forest - here's what that means:

Imagine asking 100 different experts to predict energy usage. Each expert sees a slightly different view of the data and makes their own prediction. The final answer is the average of all 100 predictions. That's Random Forest - a "forest" of decision "trees" that vote together.

Why 3 Separate Models? - **Grid model** - for Grid-powered wells (only electricity) - **Diesel model** - for Diesel and DGB wells - **CNG model** - for Turbine and DGB wells

Different fuel types behave differently, so we train a specialist for each.

Understanding the Fleet Types:

Fleet Type	What Fuel It Uses
Grid	Only electricity
Diesel	Only diesel fuel
Turbine	Only CNG (natural gas)
DGB (Dual-fuel)	BOTH diesel AND CNG

Important: DGB wells produce TWO predictions - one for Diesel, one for CNG. That's why 50 wells become 63 rows in the output.

Metrics to Understand: - **R² (R-squared):** How much of the variation the model explains. 0.80 = explains 80%. Higher is better. - **RMSE:** Average prediction error. Lower is better. - **MAE:** Average absolute error. Lower is better.

Step 6: Uncertainty Quantification

What happens: We measure how confident we are in each prediction.

No prediction is perfect. The hackathon doesn't just want your best guess - they want to know the RANGE of possible values.

Our Method - Residual Bootstrapping: 1. Calculate $\text{Actual} - \text{Predicted}$ 2. For each new prediction, randomly sample 100 past errors 3. Add each sampled error to the prediction to create 100 possible outcomes

Analogy: A weather forecast says 70°F but also $\text{range: } 65\text{-}75^{\circ}\text{F}$. The range acknowledges uncertainty. We're doing the same for energy predictions.

Step 7: Generate Predictions

What happens: We apply our trained models to the 50 test wells and create the submission file.

The Output File (solution.csv):

Column	What It Is
Masked Well Name	Identifier for each well
Fuel Type	Grid, Diesel, or CNG
Fuel Value	Our best prediction (point estimate)
Real_1 through Real_100	100 possible outcomes representing uncertainty

Important: The columns are named Real_1, Real_2, ..., Real_100 (not R_1).

Analogy: Instead of saying $\text{"this well will use exactly 50,000 gallons"}$, we say $\text{"our best guess is 50,000, but it could realistically be anywhere from 45,000 to 55,000"}$ - and we show 100 examples from that range.

Step 8: Quick Start Guide

What it is: A reference page summarizing how to run through the entire workflow quickly on hackathon day.

This step doesn't do any processing - it's just instructions for rapid execution when you're under time pressure.

Step 9: AI ML Assistant

What it is: A chat interface where you can ask questions about:

- **The workflow:** $\text{"Walk me through how this app works"}$
- **Each step:** $\text{"What does Step 5 do?"}$
- **ML concepts:** $\text{"What is cross-validation?"}$
- **Troubleshooting:** $\text{"Why is my model performing poorly?"}$
- **Local installation:** $\text{"How do I run this on my laptop?"}$
- **Our innovation:** $\text{"Why is this better than Spotfire?"}$

This is your innovation differentiator! Other teams build static pipelines. Yours can THINK and ADAPT.

Example questions to ask: - $\text{"What features should I engineer from this data?"}$ - $\text{"Would XGBoost work better than Random Forest here?"}$ - $\text{"How do I interpret this correlation matrix?"}$ - $\text{"Walk me through the whole workflow step by step"}$

Running This On Your Own Computer

If you want to run this app on your personal laptop instead of Replit:

1. Requirements: - Python 3.8 or higher - pip (Python package manager)

2. Install Dependencies: Open terminal/command prompt and run:

```
pip install streamlit pandas numpy scikit-learn matplotlib seaborn plotly openai
```

3. Set Up OpenAI API Key (for AI Assistant only): - Get an API key from <https://platform.openai.com/api-keys> - Windows: `set OPENAI_API_KEY=your-key-here` - Mac/Linux: `export OPENAI_API_KEY=your-key-here` - Note: Steps 1-7 work WITHOUT an API key. Only Step 9 needs it.

4. Run the App:

streamlit run app.py

This opens the app in your browser at <http://localhost:8501>

Quick Glossary

Term	Plain English Meaning
Feature	A column in your data - something you measure about each well
Target	What you're trying to predict (Grid kWh, Diesel gal, CNG MMBTU)
Training	Teaching the model using data where you know the answers
Prediction	The model's guess for data where you don't know the answer
Overfitting	When a model memorizes training data but fails on new data
Cross-Validation	Testing the model multiple ways to ensure consistency
Correlation	How strongly two things move together (0=none, 1=perfect)
Residual	The error: Actual value minus Predicted value
Bootstrapping	Randomly sampling from your data to estimate uncertainty
R² (R-squared)	% of variation explained by the model (0.8 = 80%)
RMSE	Average error size (in same units as target)
Imputation	Filling in missing values with reasonable estimates
Categorical	Data that's a label/category, not a number (e.g., "Grid")
Encoding	Converting categories to numbers so the model understands

Summary: What Makes This Special

1. **Predictive, not just descriptive** - We tell you what WILL happen, not just what happened
 2. **Uncertainty quantification** - 100 realizations show the range of possibilities
 3. **AI-powered** - The assistant can explain, adapt, and guide users
 4. **Free and portable** - No expensive licenses, runs anywhere with Python
 5. **Domain-aware** - Built specifically for energy/oil & gas operations
-

Created for Team Energy Gladiators - Good luck at the hackathon! 🍀†